MAH 论文的 MATLAB 求解准备材料

一份给一年级博士生也能读懂的完整说明

2026 年 4 月 10 日

目录

1 文件目的与适用边界 **4**

2 与当前主文稿的对应关系 **4**

3 为什么这篇文章优先用 MATLAB **5**

3.1 为什么不是 Dynare 5

3.2 为什么 MATLAB 合适 5

4 模型对象：先把经济学翻译成算法对象 **6**

4.1 状态、控制、价格与 payoff 对象 6

4.2 为什么 baseline 用一维 z 6

5 为什么静态生产块要先优化掉 **7**

5.1 经济学直觉 7

5.2 一个最小例子 7

6 当前模型下，Bellman 到底在解什么 **8**

6.1 Type-B 的 route choice 是当前模型的中心 8

6.2 三个类型的流量收益 9

6.3 Bellman 系统 9

7 你在 MATLAB 里真正做的，是一个嵌套固定点 **9**

7.1 最核心的一句话 9

7.2 内层固定点：给定价格，解企业问题与稳态分布 10

7.3 外层固定点：更新价格直到市场清算 10

8 详细算法：一步一步写给初学者看 **10**

8.1 Step 0：先固定一个最小可运行版本 10

8.2 Step 1：离散化状态空间 11

8.3 Step 2：猜测价格 11
8.4 Step 3：计算静态块和流量收益 11
8.5 Step 4：初始化 value functions 12
8.6 Step 5：做 value function iteration 12
8.7 Step 6：恢复 policies 12
8.8 Step 7：构造总转移矩阵 13
8.9 Step 8：进入与 entrant distribution 13
8.10 Step 9：求稳态分布 13
8.11 Step 10：计算 market-clearing residuals 14
8.12 Step 11：更新价格 14

9 MATLAB 文件结构建议 14
9.1 各文件做什么 15

10 给初学者的伪代码 16

11 SMM / indirect inference：在求解器外面再包一层 16
11.1 最核心的直觉 16
11.2 参数分块 17
11.3 数据到位后最自然的 moments 分组 17
11.4 SMM 目标函数 17
11.5 为什么你现在还不急着写 SMM 代码 18

12 反事实：为什么要区分 PE 与 GE 18
12.1 Partial equilibrium counterfactual 18
12.2 General equilibrium counterfactual 18

13 福利分析：第一版怎么做最稳 19
13.1 不要把创新数量机械等同于福利 19
13.2 第一版最稳的 welfare 对象 19

14 初学者最容易踩的坑 19
14.1 坑 1：一开始就把模型写太大 19
14.2 坑 2：把静态块和动态块混在一起 20
14.3 坑 3：Bellman 收敛了，但分布没对 20
14.4 坑 4：价格更新震荡 20
14.5 坑 5：先写 SMM，后写 solver 20

15 推荐实施顺序 20

16 一句话总结这整套技术逻辑 21

A 附录 A：最小函数接口示意 **21**

B 附录 B：当前阶段最推荐的任务拆分 **22**

1 文件目的与适用边界

这份材料的目标不是替代正文，也不是提前伪造 Section 5 的经验结果，而是把你在数据到位之后真正要做的数值工作完整拆开，形成一份**可以直接交给一年级博士生阅读、并可据此开始写 MATLAB 求解器**的技术准备说明。

它只服务四件事：

1. 把当前主模型翻译成可计算的动态行业均衡问题；
2. 解释为什么这篇文章适合用 MATLAB 而不是 Dynare 做第一版求解；
3. 说明 Bellman、稳态分布、市场清算、SMM、反事实和福利分析之间的技术衔接；
4. 给出一套可执行的 MATLAB 文件架构、伪代码和调试顺序。

这份材料**不做**下面这些事情：

- 不报告任何未经数据支持的经验结果；
- 不虚构参数值、校准值或估计值；
- 不把未来可能扩展的二维状态空间直接塞进当前 baseline；
- 不把旧版本的宽泛政策原语重新带回当前主模型。

因此，这是一份 **solver blueprint / implementation note**，而不是 results note。

2 与当前主文稿的对应关系

当前主文稿已经把模型对象锁定为：

- 三类组织状态 $s \in \{A, B, C\}$；
- 一维持久能力状态 $z \in Z \subset \mathbb{R}_+$；
- 两个价格对象 w 和 p_m；
- 静态生产块先优化掉，再把优化后的流量收益放入 Bellman；
- Type-B 企业的核心问题是 **commercialization assignment**；
- MAH 通过 route-specific external commercialization friction τ_{ext} 进入模型；
- 行业闭合方式是 entry / exit / switching / stationary equilibrium。

所以，这份材料的数值对象也必须完全服从这套结构。用一句话说：

> 你在 MATLAB 里真正要求解的，不是一个代表性企业的局部最优问题，而是一个带有组织状态、进入退出、委托生产市场与稳态分布的**异质企业动态行业均衡**。

3 为什么这篇文章优先用 MATLAB

3.1 为什么不是 Dynare

Dynare 最擅长的是：

- 代表性家庭 / 代表性企业；
- 一阶条件系统；
- 理性预期均衡；
- 稳态附近的线性化或高阶近似；
- 标准 DSGE、RBC、新凯恩斯模型。

而你现在这篇文章的第一版核心任务不是这些。你现在的主任务是：

- 给定价格，解三个组织状态的 Bellman 方程；
- 恢复最优 policy functions；
- 把 policy 变成行业总转移矩阵；
- 求 stationary distribution；
- 让 contract-manufacturing market 和 labor market 清算；
- 再把整个求解器包进 SMM / indirect inference 的参数搜索外层。

这类问题的关键词是：**dynamic programming, fixed point, distribution evolution, market clearing**。因此，第一版最稳的工具组合仍然是：

- 经验部分：`Stata`
- 理论模型与数值求解：`MATLAB`

3.2 为什么 MATLAB 合适

MATLAB 适合你的原因不是“它最先进”，而是：

- 大多数异质企业动态模型课程讲义和旧代码都用 MATLAB；
- 它对矩阵、离散状态转移、Bellman iteration 和 root-finding 很自然；
- 对第一次真正做动态行业模型的人，它比 Dynare 的学习路径更直观。

如果以后你把 baseline 升级成二维状态、非平滑 choice probability、更多市场清算方程、或者需要更大规模的并行计算，Julia 是很自然的下一步。但在当前阶段，MATLAB 是更稳的第一步。

4 模型对象：先把经济学翻译成算法对象

4.1 状态、控制、价格与 payoff 对象

为了真正写出求解器，必须先把所有对象分清楚。下面这张表是最小而完整的基准分类。

对象	类型	解释
$s \in \{A, B, C\}$	离散状态	企业当前组织形式
$z \in Z$	连续状态	一维综合能力状态
$M \in \{0, 1\}$	制度状态	pre-MAH / post-MAH
w	价格	工资或综合要素价格
p_m	价格	合格委托生产服务价格
n, k, m	静态投入	labor, capital services, materials
x_A, x_B	动态控制	A/B 型企业的 idea-generation intensity
$h \in H$	动态控制	B 型企业对单个 idea 的 commercialization route
$j \in J(s)$	动态控制	维持、切换组织形式或退出
κ_{sj}	成本对象	组织切换成本
τ_{ext}	制度摩擦	external commercialization 的治理/合规摩擦
$\bar{\pi}_A, \bar{\pi}_C$	优化后静态收益	当期经营利润
Ψ_B	每个 idea 的净值	B 型 idea 的 commercialization value
Π_A, Π_B, Π_C	流量收益对象	Bellman 方程的当期收益输入
V_A, V_B, V_C	价值函数	三类组织状态下的 continuation value
μ_A, μ_B, μ_C	分布对象	行业稳态分布

4.2 为什么 baseline 用一维 z

当前 baseline 只用一维 z，并不是说企业只有一个真实能力，而是说：

> 在正文的第一版里，用一个一维 latent capability 来汇总 research、project management、regulatory execution、external coordination 与 commercialization capability，是最稳的可计算基线。

更具体地说，z 可以理解为 research-commercialization capability。这个对象不是纯 TFP，也不是纯研发效率；它是**把 idea 变成 realized innovation** 所需的综合能力。

二维扩展

$$z = (z_R, z_C)$$

当然是可以保留接口的，其中 z_R 表示 research capability，z_C 表示 commercialization / execution capability。但这一扩展留在附录和未来版本更合适，因为它会显著放大状态空间和估计难度。

5 为什么静态生产块要先优化掉

5.1 经济学直觉

你的论文真正要识别的是：

> MAH 是否改变 original-drug ideas 从 invention 到 commercialization 的组织路线。

因此，Bellman 的核心必须围绕：

- 研发强度；
- commercialization route；
- 组织选择；
- 进入与退出。

如果 labor、capital、materials、当期生产量既不成为下一期状态，也没有动态调整成本，那么它们首先只影响**当期经营利润**。这时最自然的写法是：

1. 先在给定 (s, z, w, p_m) 下把静态投入优化掉；
2. 得到优化后的 $\bar{\pi}_A$ 和 $\bar{\pi}_C$；
3. 再把这些对象放进动态 Bellman。

5.2 一个最小例子

以 integrated firm 为例，原始当期利润可以写成：

$$\pi_A(z; n, k, m, x_A) = p_A q_A(n, k, m; z) - wn - rk - c_m m + x_A \Omega_A(z) - \frac{\chi_A}{\eta} x_A^\eta - f_A.$$

如果 n, k, m 都不成为明天的状态变量，那么先解：

$$\bar{\pi}_A(z; w, r, c_m) = \max_{n,k,m} \left\{ p_A q_A(n, k, m; z) - wn - rk - c_m m \right\}.$$

然后再写：

$$\Pi_A(z; M, w, p_m) = \bar{\pi}_A(z; w, r, c_m) + \max_{x_A \geq 0} \left\{ x_A \Omega_A(z) - \frac{\chi_A}{\eta} x_A^\eta \right\} - f_A.$$

Type-C 企业同理：

$$\bar{\pi}_C(z;p_m,w,r,c_m)=\max_{q_m,n,k,m}\left\{p_mq_m-c_C(q_m,n,k,m;z)-wn-rk-c_mm-f_C\right\}.$$

这一步不是技术偷懒，而是让 Bellman 只保留真正改变未来状态和未来价值的动态边际。

6 当前模型下，Bellman 到底在解什么

6.1 Type-B 的 route choice 是当前模型的中心

当前版本最关键的 block 不是 $A/B/C$ 身份标签本身，而是 Type-B 企业如何把一个 research-originated idea 分配到不同 commercialization routes 上。

设可行 route 集合为：

$$H=\{ext,int,tr,ab\},$$

其中：

- ext：外部商业化，即通过合格外部厂商完成 commercialization；
- int：内部整合式实现；
- tr：项目转让；
- ab：放弃。

外部商业化路线的净值是：

$$H_B^{ext}(z,p_m;M)=\lambda_{ext}(z,p_m)R_B(z)-p_m-\tau_{ext}(M). \tag{1}$$

内部实现路线可以写成：

$$H_B^{int}(z)=\lambda_{int}(z)R_B(z)-\kappa_I(z). \tag{2}$$

转让路线和放弃路线分别写成：

$$H_B^{tr}(z)=T_B(z),\qquad H_B^{ab}(z)=0.$$

因此，每个 idea 的净值是：

$$\Psi_B(z;p_m,M)=\max\left\{H_B^{ext}(z,p_m;M),H_B^{int}(z),H_B^{tr}(z),0\right\}. \tag{3}$$

这一步把制度机制锁定在当前有效主语上：

$$\frac{\partial H_B^{ext}}{\partial\tau_{ext}}=-1,\qquad \frac{\partial H_B^{int}}{\partial\tau_{ext}}=0,\qquad \frac{\partial H_B^{tr}}{\partial\tau_{ext}}=0.$$

也就是说，MAH 不是一个 generic policy dummy，而是 external commercialization 这一路线的 route-specific friction 的下降。

6.2 三个类型的流量收益

当前 baseline 下，三类企业的优化后流量收益可以写成：

$$\Pi_A(z; M, w, p_m) = \bar{\pi}_A(z; w, r, c_m) + \max_{x_A \geq 0} \left\{ x_A \Omega_A(z) - \frac{\chi_A}{\eta} x_A^\eta \right\}, \tag{4}$$

$$\Pi_B(z; M, p_m) = -f_B + \max_{x_B \geq 0} \left\{ x_B \Psi_B(z; p_m, M) - \frac{\chi_B}{\eta} x_B^\eta \right\}, \tag{5}$$

$$\Pi_C(z; M, w, p_m) = \bar{\pi}_C(z; p_m, w, r, c_m). \tag{6}$$

在很多数值实现里，如果 $\eta > 1$ 且存在 interior solution，上面两个 innovation intensity 问题可以先用闭式解代回；如果没有简单闭式，也可以做一维数值优化。

6.3 Bellman 系统

设 $J(s)$ 是当前组织状态 s 的可行下一期组织选择集合。对任意当前状态 (s, z) 和候选下一期状态 j，定义：

$$W_{sj}(z; M) = \Pi_j(z; M, w, p_m) - \kappa_{sj}(z; M) + \beta \mathbb{E}\left[V_j(z'; M) \mid z, j \right]. \tag{7}$$

则 Bellman 方程是：

$$V_s(z; M) = \max \left\{ 0, \max_{j \in J(s)} W_{sj}(z; M) \right\}, \qquad s \in \{A, B, C\}. \tag{8}$$

这里的 0 表示 exit option。当前组织选择不是为组织本身而组织，而是因为不同组织形态意味着：

- 不同流量收益；
- 不同 commercialization routes；
- 不同 continuation values。

7 你在 MATLAB 里真正做的，是一个嵌套固定点

7.1 最核心的一句话

当前求解器最重要的技术直觉是：

> **你不是直接在一次循环里同时得到价格、价值函数和分布**。你做的是：给价格 → 解 Bellman → 得 policy → 算分布 → 检查市场清算 → 更新价格。

也就是说，这是一个**嵌套固定点**。

7.2 内层固定点：给定价格，解企业问题与稳态分布

给定猜测价格 (w, p_m) 和制度状态 M，你需要完成下面几步：

1. 离散化 z 并给定转移矩阵 P_z；
2. 计算 Π_A, Π_B, Π_C；
3. 迭代求解 V_A, V_B, V_C；
4. 恢复最优 policy：$x_A^*, x_B^*, h_B^*, j_s^*$；
5. 构造总转移矩阵；
6. 结合 entrant inflow，求稳态分布 μ_A, μ_B, μ_C；
7. 由稳态分布和 policy 计算 excess demand / supply。

7.3 外层固定点：更新价格直到市场清算

如果稳态分布下的 demand 与 supply 不相等，就更新价格。例如：

$$D_m(p_m; M) - S_m(p_m, w; M) \neq 0$$

时，需要更新 p_m；如果 labor market 也内生，则还需要更新 w。外层迭代直到两组 market-clearing conditions 同时满足。

8 详细算法：一步一步写给初学者看

下面把整个求解过程拆成最细版本。

8.1 Step 0：先固定一个最小可运行版本

在真正写代码之前，先把第一版范围锁死。最稳的第一版是：

1. 一维状态 z；
2. 三类组织状态 $A/B/C$；
3. 两个价格 w, p_m；
4. hard-max Bellman，不上平滑 logit；
5. 先做 stationary comparison，不做 transition path；

6. 先不做 SMM，只做 hand-calibrated pre/post comparison。

这是第一版最稳的原因很简单：你先要有一个能跑的 baseline solver，才能在其外面包 estimation 和 counterfactual。

8.2 Step 1：离散化状态空间

设离散 grid 为：

$$\{z_1, \ldots, z_{N_z}\}.$$

同时给出能力状态转移矩阵：

$$P_z(i, i') = \Pr\{z' = z_{i'} \mid z = z_i\}.$$

最简单的基准写法是所有组织状态共用一个 P_z；更丰富一点的写法是用 P_z^A, P_z^B, P_z^C。如果你第一次写，建议先共用同一个。

8.3 Step 2：猜测价格

给一组初始价格：

$$(w^{(0)}, p_m^{(0)}).$$

如果你想先让 w 外生，只更新 p_m，也是完全合法的第一步。等求解器跑稳后再把 labor market 关上。

8.4 Step 3：计算静态块和流量收益

在每个 z_i 上分别计算：

- $\bar{\pi}_A(z_i; w)$；
- $H_B^{ext}(z_i, p_m; M), H_B^{int}(z_i), H_B^{tr}(z_i)$；
- $\Psi_B(z_i; p_m, M)$；
- $\bar{\pi}_C(z_i; p_m, w)$；
- 最终得到 $\Pi_A(z_i), \Pi_B(z_i), \Pi_C(z_i)$。

这一层结束后，你实际上已经把“制度机制”翻译成了数值输入：τ_{ext} 只进入 H_B^{ext}，从而进入 Ψ_B，再进入 Π_B。

8.5 Step 4：初始化 value functions

给一个初始猜测：

$$V_A^{(0)}(z_i) = 0, \qquad V_B^{(0)}(z_i) = 0, \qquad V_C^{(0)}(z_i) = 0,$$

或者用静态利润除以 $(1-\beta)$ 作为更聪明的初始值。

8.6 Step 5：做 value function iteration

对每个 z_i，先算 continuation values，例如：

$$E[V_A(z') \mid z_i] = \sum_{i'} P_z(i, i') V_A(z_{i'}),$$

其余两个类型同理。

然后对每个当前组织状态分别构造所有 choice-specific values。例如对当前为 B 的企业：

$$\begin{aligned}
W_{BB}(z_i) &= \Pi_B(z_i) + \beta E[V_B(z') \mid z_i], \\
W_{BA}(z_i) &= \Pi_A(z_i) - \kappa_{BA}(z_i; M) + \beta E[V_A(z') \mid z_i], \\
W_{BC}(z_i) &= \Pi_C(z_i) - \kappa_{BC}(z_i; M) + \beta E[V_C(z') \mid z_i], \\
W_{BX}(z_i) &= 0.
\end{aligned}$$

再更新：

$$V_B^{new}(z_i) = \max\{W_{BB}(z_i), W_{BA}(z_i), W_{BC}(z_i), 0\}.$$

对 A 和 C 重复同样操作。然后用 sup norm 判断是否收敛：

$$\max\left\{\|V_A^{new} - V_A\|_\infty, \|V_B^{new} - V_B\|_\infty, \|V_C^{new} - V_C\|_\infty\right\} < \varepsilon.$$

8.7 Step 6：恢复 policies

Bellman 收敛后，你需要恢复：

- 每个组织状态在每个 z_i 上最优 stay / switch / exit 的选择；
- $x_A^*(z_i), x_B^*(z_i)$；
- Type-B 的最优 commercialization route $h_B^*(z_i)$；
- entrant 的最优进入类型。

如果 Bellman 用的是 hard max，这一步非常简单：保留哪一个 W_{sj} 最大即可。

8.8 Step 7：构造总转移矩阵

行业的联合状态空间是：

$$(s, z) \in \{A, B, C\} \times \{z_1, \ldots, z_{N_z}\}.$$

因此总状态数是 $3N_z$。你需要构造一个大小为 $(3N_z) \times (3N_z)$ 的矩阵 T，使得：

$$\mu' = T\mu + e.$$

这里 e 是 entrants inflow。
构造方法是逐行填。例子：

- 如果当前状态是 (B, z_i)，policy 说“留在 B”，那么这一行把概率质量分配到未来的 $(B, z_{i'})$，权重是 $P_z(i, i')$；
- 如果 policy 说“转到 A”，就分配到 $(A, z_{i'})$；
- 如果 policy 说“退出”，这一行全为 0。

8.9 Step 8：进入与 entrant distribution

给 entrant 初始能力分布 G_0。离散化后记作向量 g_0。则进入某类组织类型的期望净值是：

$$\mathcal{V}_{Ej}(M) = \sum_i g_0(i) V_j(z_i; M) - c_{Ej}.$$

然后你需要规定 entrant rule。最简单的三种写法是：

1. 只允许进入期望值最高且非负的类型；
2. 做 softmax / logit entry shares；
3. 对潜在 entrant 加 idiosyncratic entry cost，形成向上倾斜的 entry supply。

第一版最简单的是第 1 种，等以后做更真实的 entry mass response 时，再换成第 2 或第 3 种。

8.10 Step 9：求稳态分布

最稳的办法是前向迭代。给一个初始分布 $\mu^{(0)}$，重复：

$$\mu^{(n+1)} = T\mu^{(n)} + e,$$

直到收敛为止。
最后将联合分布拆回：

$$\mu_A, \quad \mu_B, \quad \mu_C.$$

这样你就得到了行业的 stationary distribution。

8.11 Step 10：计算 market-clearing residuals

当前 baseline 最重要的是 contract-manufacturing market：

$$D_m(p_m; M) = S_m(p_m, w; M).$$

一个自然的离散写法是：

$$D_m(p_m; M) = \sum_i \mu_B(z_i)\, x_B^*(z_i; p_m, M)\, \mathbf{1}\{h_B^*(z_i) = ext\}, \tag{9}$$

$$S_m(p_m, w; M) = \sum_i \mu_C(z_i)\, q_m^*(z_i; p_m, w). \tag{10}$$

如果 labor market 也内生，则再算：

$$L^d(w, p_m; M) - \bar{L}.$$

8.12 Step 11：更新价格

最简单的写法是阻尼更新：

$$p_m^{new} = p_m^{old} + \omega_m\big(D_m - S_m\big), \tag{11}$$

$$w^{new} = w^{old} + \omega_w\big(L^d - \bar{L}\big), \tag{12}$$

其中 ω_m, ω_w 是小的 damping parameters。

如果只有一个市场清算方程，也可以用 `fzero`。如果两个价格一起更新，则可以用 `fsolve`，但第一版往往阻尼更新更稳。

9 MATLAB 文件结构建议

下面是一套非常稳的文件架构。你不需要一次写得很花，只要结构清楚即可。

```
main_run.m
set_params.m
make_grids.m
static_payoffs.m
idea_values_B.m
solve_bellman.m
recover_policies.m
entry_block.m
```

```
build_transition.m
stationary_dist.m
market_clearing.m
solve_equilibrium.m
moments_from_equilibrium.m
objective_smm.m
run_counterfactuals.m
welfare_block.m
```

9.1 各文件做什么

set_params.m 存放参数、容忍度、最大迭代次数、外层更新步长。

make_grids.m 生成 `zgrid`、`Pz`、entrant distribution 等。

static_payoffs.m 输入 $(zgrid, w, p_m, M, params)$，输出 $\bar{\pi}_A, \bar{\pi}_C$ 以及必要的 static decisions。

idea_values_B.m 只做 Type-B 的 idea-level block：

$$H_B^{ext},\ H_B^{int},\ H_B^{tr},\ \Psi_B,\ x_B^*.$$

solve_bellman.m 输入三个流量收益向量和转移矩阵，输出 V_A, V_B, V_C。

recover_policies.m 从 choice-specific values 中恢复 stay/switch/exit、route choice 等 policies。

entry_block.m 算 entrant values、entry composition 和 entrants inflow。

build_transition.m 把 policy + P_z 变成联合状态总转移矩阵 T。

stationary_dist.m 用前向迭代或解线性方程的方法算稳态分布。

market_clearing.m 给定分布和 policy，算 contract manufacturing market 和 labor market 的 excess demand。

solve_equilibrium.m 这是核心函数。给定参数和制度状态 M，输出整套平稳均衡对象。

objective_smm.m 把 `solve_equilibrium` 包起来，输出 moment distance。

run_counterfactuals.m 做 PE / GE 反事实。

welfare_block.m 从 equilibrium objects 出发计算福利或 industry surplus。

10 给初学者的伪代码

下面这段伪代码应该反复读，直到真正能在脑子里跑起来。

```
Given parameters theta and policy state M:

1. Make z-grid and transition matrix Pz.
2. Guess prices (w, pm).
3. Repeat outer loop until prices clear markets:
   3.1 Compute optimized static payoffs for A and C.
   3.2 Compute B-type route values and idea values.
   3.3 Build flow payoffs PiA, PiB, PiC.
   3.4 Solve Bellman system for VA, VB, VC.
   3.5 Recover policies: xA, xB, route choice, stay/switch/exit.
   3.6 Compute entrant values and entrant inflow.
   3.7 Build full transition matrix T.
   3.8 Iterate forward to stationary distribution mu.
   3.9 Compute excess demand in manufacturing and labor markets.
   3.10 Update prices.
4. Return equilibrium objects.
```

如果你能把上面这十步顺着说清楚，你就已经真正明白这篇文章的数值部分在做什么了。

11 SMM / indirect inference：在求解器外面再包一层

11.1 最核心的直觉

上面整套步骤回答的是：

> 给定参数，模型能推出什么均衡？

而 SMM 回答的是：

> 参数未知时，什么参数最能让模型推出的数据矩与真实数据矩相接近？

因此，SMM 只是把整个 equilibrium solver 包进一个 parameter search outer layer。

11.2 参数分块

你未来最自然的参数分块大致有五组：

1. state process block：z 的转移和 entrant initial distribution；
2. organization block：κ_{sj}、exit-related objects；
3. commercialization block：τ_{ext}、λ_{ext}、route-specific payoffs；
4. production block：$\bar{\pi}_A, \bar{\pi}_C$ 的成本参数；
5. innovation block：χ_A, χ_B, η 及相关 payoff parameters。

11.3 数据到位后最自然的 moments 分组

当前草稿已经确定了经验 hierarchy。最自然的 moments 可以分成四组：

1. pre-reform organizational structure；
2. pre-reform producer-side equilibrium；
3. post-reform commercialization and innovation outcomes；
4. validation moments not directly targeted。

更具体地说，最自然的 targeted moments 会围绕：

- $A/B/C$ shares；
- entry composition；
- organizational transitions；
- entrusted production / external commercialization proxies；
- original-drug counts and shares；
- producer-side outcomes of type-C firms。

11.4 SMM 目标函数

标准写法是：

$$Q(\theta) = \left(m^{model}(\theta) - m^{data}\right)' W \left(m^{model}(\theta) - m^{data}\right).$$

其中：

- θ 是参数向量；

- $m^{model}(\theta)$ 是用求解器推出的 model moments；
- m^{data} 是数据 moments；
- W 是权重矩阵。

数值实现上，`objective_smm.m` 做的就是：

1. 输入参数 θ；
2. 分别求解 $M = 0$ 和 $M = 1$ 下的平稳均衡；
3. 计算 model moments；
4. 返回加权距离。

11.5 为什么你现在还不急着写 SMM 代码

因为在没有数据之前，最重要的是先保证 baseline equilibrium solver 稳定可运行。经验估计永远是建立在求解器已经可靠的前提上的。如果 baseline solver 还没跑稳，一上来写 SMM 通常只会放大 bug。

12 反事实：为什么要区分 PE 与 GE

12.1 Partial equilibrium counterfactual

PE 的做法是：固定价格和分布，只改变

$$\tau_{ext}(0) \to \tau_{ext}(1).$$

它回答的问题是：

如果不允许价格、entry 和行业分布反馈，MAH 的直接机制链到底多强？

所以 PE 最适合看：

$$\tau_{ext} \downarrow \Rightarrow H_B^{ext} \uparrow \Rightarrow \Psi_B \uparrow \Rightarrow x_B \uparrow \Rightarrow V_B \uparrow .$$

12.2 General equilibrium counterfactual

GE 则允许：

- entry 调整；
- stationary distribution 调整；
- p_m 调整；

- w 调整；
- producer-side outcomes 调整。

它回答的是：

> 当所有相关市场和组织分布都内生响应后，MAH 的最终行业均衡效应是什么？

当前主文稿已经非常强调这一点：producer-side outcomes 在 GE 下并不必然单调。所以未来做 quantitative work 时，**PE intuition 和 GE results 必须分开报告**。

13 福利分析：第一版怎么做最稳

13.1 不要把创新数量机械等同于福利

当前框架下，一个非常重要的纪律是：innovation counts 的上升，不自动等于 welfare 的上升。因此，第一版 welfare block 不应偷懒地写成“更多 original-drug products = 更高 welfare”。

13.2 第一版最稳的 welfare 对象

如果你当前只把模型闭合到 industry level，那么最稳的第一版 welfare 写法是 industry surplus / net value comparison：

$$\mathcal{W}(M) = \text{operating surplus} + \text{innovation value} - \text{entry costs} - \text{switching costs} - \text{governance/compliance co}$$

然后比较：

$$\Delta\mathcal{W} = \mathcal{W}(1) - \mathcal{W}(0).$$

如果以后你加了更完整的 household side 和 demand closure，再升级到 consumption-equivalent welfare 也完全可以。但第一版没有必要一上来就背这个负担。

14 初学者最容易踩的坑

下面这些坑，第一次写异质企业动态模型的人几乎一定会踩。

14.1 坑 1：一开始就把模型写太大

最常见错误是：一上来就想同时做二维状态、平滑 choice probabilities、完全内生的 entry mass、两套 market clearing、SMM、反事实和 welfare。结果往往是没有一个模块先跑稳。

建议：先只做一维 z、hard max、stationary comparison。

14.2 坑 2：把静态块和动态块混在一起

如果你把 labor、capital、materials、research intensity、route choice、organization choice 全塞进一个大 Bellman，数值上会变得很乱，而且经济解释也会跑偏。

建议：永远先确认哪个变量只是当期投入，哪个变量真的决定未来状态。

14.3 坑 3：Bellman 收敛了，但分布没对

有时 value function 看起来已经稳定，但总转移矩阵写错了，stationary distribution 完全不对。

建议：每一步都做守恒检查：

- transition matrix 的每一行概率有没有加总到 1 或者小于等于 1；
- exit 行是不是全 0；
- entrant inflow 有没有正确加进来；
- stationary distribution 是否非负且总质量合理。

14.4 坑 4：价格更新震荡

外层 fixed point 如果更新太激进，很容易震荡不收敛。

建议：用阻尼更新，小步走；必要时先只关掉一个市场清算条件。

14.5 坑 5：先写 SMM，后写 solver

这通常是本末倒置。SMM 只是求解器外面的一层。没有稳定 solver，就没有可靠 SMM。

15 推荐实施顺序

如果现在真的让一个一年级博士生从零开始实现，我建议顺序是：

1. 先只写 `make_grids.m`；
2. 再写 `static_payoffs.m` 和 `idea_values_B.m`；
3. 再写 `solve_bellman.m`；
4. 再写 `recover_policies.m`；
5. 再写 `build_transition.m`；
6. 再写 `stationary_dist.m`；

7. 再写 `market_clearing.m`；
8. 最后再把它们包成 `solve_equilibrium.m`。

只有当 `solve_equilibrium.m` 稳定以后，才开始：

1. 写 `moments_from_equilibrium.m`；
2. 写 `objective_smm.m`；
3. 写 `run_counterfactuals.m`；
4. 写 `welfare_block.m`。

16 一句话总结这整套技术逻辑

你在 MATLAB 里做的整件事，本质上是：

> 把“企业在不同组织状态和能力状态下的最优动态选择”翻译成 Bellman 系统；再把这个 Bellman 系统翻译成行业状态转移矩阵；再把这个转移矩阵和 entrants 一起翻译成稳态分布；再让价格调整到 contract-manufacturing market 与 labor market 清算；最后把这一整套平稳均衡求解器包进 SMM、counterfactual 与 welfare analysis 的外层。

只要你把这条链真正想通，MATLAB 语法本身反而是次要的。真正难的不是写 `for` 循环，而是清楚知道：哪一个对象先算，哪一个对象后算，哪些对象是内层固定点，哪些对象是外层固定点，以及当前文章真正的核心机制到底是哪一个数学对象。

A 附录 A：最小函数接口示意

下面是一套非常简洁但足够实用的函数接口示意。

```
function params = set_params()
function grids  = make_grids(params)
function static = static_payoffs(params, grids, prices, M)
function bblock = idea_values_B(params, grids, prices, M)
function bell   = solve_bellman(params, grids, flowpay, prices, M)
function pol    = recover_policies(params, grids, bell, flowpay, prices, M)
function entry  = entry_block(params, grids, bell, M)
function T      = build_transition(params, grids, pol, entry, M)
function mu     = stationary_dist(params, T, entry)
function mc     = market_clearing(params, grids, pol, mu, prices, M)
function eq     = solve_equilibrium(params, grids, M)
```

```
function moms   = moments_from_equilibrium(eq, params)
function obj    = objective_smm(theta, data_moments, W)
function cf     = run_counterfactuals(eq0, params)
function wel    = welfare_block(eq0, eq1, params)
```

B 附录 B：当前阶段最推荐的任务拆分

为了避免项目失控，当前阶段最推荐的拆分是：

1. **理论封板：**把正文与 Appendix B/C 写稳；
2. **solver baseline：**只做 hand-calibrated equilibrium solver；
3. **数据 construction：**等数据准备好后再进入 moments；
4. **估计：**最后再上 SMM / indirect inference。

这是最符合当前文章节奏的推进方式。